

INFO5995 Assignment 1 Report

Weihaio Cheng, Yucheng Zhao, Xu He, Yibo Tu, and Yuchen Zhou

1 Introduction

This report analyzes the provided Android APK to identify a security-relevant vulnerability. APK decompilation is necessary because Android apps are distributed in compiled form, and reverse engineering is required to inspect their internal logic and security-sensitive code. We used AI assistance to select JADX as the decompilation tool, which allowed us to inspect the manifest and authentication-related source files for later vulnerability analysis.

2 Application Purpose And Protected Assets

This application implements a basic local registration and login system. User credentials are stored in a local file, and successful login generates a session token for access control. Logging out clears the session and terminates access. The key protected assets in this application are: 1. User credentials: Stored in a local file (`credentials.txt`), including usernames and passwords. 2. Session tokens: Generated after successful login and stored in `SharedPreferences` (labeled `sessionToken`), used to maintain authenticated access. 3. Login session state: Managed via the session token, which determines whether a user is granted access to protected features.

3 System Model

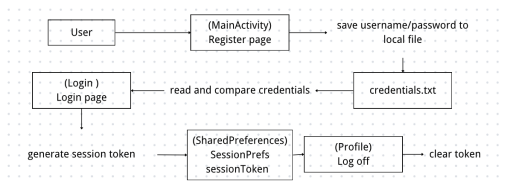


Figure 1: Simplified System Model

4 Randomness-Related Vulnerability Identification

4.1 Static Analysis Results

Static analysis of the decompiled APK identified two locations related to random value generation. The first location is `MainActivity.randomNumberGenerator()`, which uses the `java.util.Random` class to generate an integer between 0 and 99. The second location is `Login.generateSessionToken()`, which also uses `java.util.Random`, but generates a 16-character alphanumeric string intended to serve as a session token.

4.2 Security Role Analysis

The value generated in `MainActivity.randomNumberGenerator()` appears to be a general-purpose random integer. It is not clearly used in login, authorization, or session control. Therefore, it is not treated as a security-sensitive value. In contrast, the value generated in `Login.generateSessionToken()` is stored in `KEY_SESSION_TOKEN` through `createSession()` after successful login. This indicates that it is used as a session token. Since a session token serves as a session credential, it should be treated as security-sensitive.

Location	Generator API	Generated Value	Security Role
MainActivity.randomNumberGenerator()	java.util.Random	Integer between 0 and 99	General random value / UI use
Login.generateSessionToken()	java.util.Random	16-character alphanumeric string	Session token / session credential

Figure 2: Randomness-Related Locations

4.3 Core Vulnerability Determination

Not every use of `Random` constitutes a vulnerability. The key consideration is whether the generated value is used in

a security-sensitive context [1]. In this APK, the random number generated in `MainActivity` has no clear security role. However, `Login.generateSessionToken()` produces a session token that is stored and used for session management. Since `java.util.Random` is not designed for security-sensitive purposes, using it to generate a session token reduces the unpredictability of the token and weakens session security [1]. Therefore, `Login.generateSessionToken()` is identified as the core vulnerability, as it uses `java.util.Random` to generate a security-sensitive session token.

4.4 Fix and Mitigation

A suitable fix is to replace `java.util.Random` with `java.security.SecureRandom` when generating the session token. Using a stronger generator and a higher-entropy token makes the session value harder to predict.

5 Vulnerability Explanation and Threat Model

5.1 Vulnerability Analysis and Observation Requirements

The vulnerability originates from the method `generateSessionToken()` in the `Login.java` class, where a 16-character alphanumeric session token is generated using `java.util.Random`.

This token is subsequently stored via `createSession()` in local storage (e.g., `SharedPreferences`) and is used as a session identifier for maintaining authentication state.

The core issue lies in the use of `java.util.Random`, which is a general-purpose pseudo-random number generator (PRNG) and not a cryptographically secure pseudo-random number generator (CSPRNG). Its outputs are deterministic and exhibit statistical predictability when sufficient samples are observed.

As a result, the generated session token lacks sufficient entropy and unpredictability, making it unsuitable for use as a security-sensitive credential such as a session identifier.

To exploit this weakness, an attacker does not require access to the application's original source code. However, the attacker must be capable of observing or collecting multiple token samples generated over time, either through repeated interactions with the application or by monitoring relevant execution contexts.

5.2 Realistic Attacker Model

We consider a realistic adversary model involving low-privilege but capable attackers.

Attacker Types:

- **Network adversary (Man-in-the-Middle, MITM):** An attacker within the same local network (e.g., public Wi-Fi), capable of intercepting unencrypted traffic.

- **Malicious co-resident application:** A third-party application installed on the victim's device that operates without requiring root privileges.

Attacker Capabilities:

- **Zero-privilege execution:** The attacker does not require physical access to the victim's device and does not rely on bypassing Android's sandbox or obtaining root privileges.
- **Dynamic sample collection:** The attacker can collect session token samples by:
 - Intercepting network traffic (if not properly protected), or
 - Repeatedly triggering the login functionality to obtain multiple token instances.
- **Offline analysis capability:** The attacker can perform offline computation using commodity hardware or simple scripts to analyze collected token samples. Since `java.util.Random` is based on a known linear congruential generator (LCG), this may allow reduction of the effective search space or partial inference of the generator state.

5.3 Attack Scenario

The attack proceeds as follows:

Step 1: Observation and Sample Collection

The attacker repeatedly triggers the login process or observes network traffic to collect multiple session tokens generated by `generateSessionToken()`.

Step 2: PRNG Analysis

Since `java.util.Random` is not cryptographically secure and relies on a deterministic algorithm, the attacker analyzes collected token samples to identify patterns and reduce uncertainty in the generation process.

Step 3: Token Prediction

Based on the observed outputs, the attacker narrows down the possible token space or partially predicts future token values with higher probability than random guessing.

Step 4: Session Impersonation

The attacker injects a predicted or previously observed valid `sessionToken` into crafted requests or replaces the locally stored token. If the system relies solely on this token for authentication, it may accept the forged token and grant unauthorized access, resulting in session impersonation.

References

- [1] OWASP MOBILE APPLICATION SECURITY. Mastg-test-0204: Insecure random api usage. <https://mas.owasp.org/MASTG/tests/android/MASVS-CRYPTO/MASTG-TEST-0204/>, n.d. OWASP Mobile Application Security Testing Guide.